COMPUTER ENGINEERING

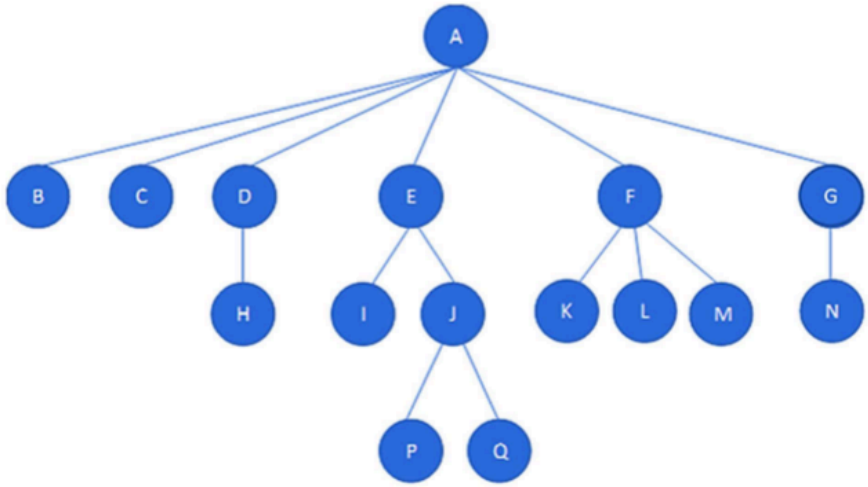
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/26	**DATE SUBMITTED:**	09/22/26
NAME:	Cruz, Elijah J	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 **Hands-on Activity 9.1 Tree ADT**

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree



TASK 1 (TABLE 9.1)
CODE

```
//Cruz, Elijah CPE21S2
#include <iostream>
#include "BST.h"

int main(){
    //root node:
    Tree<char> tree( value: 'A');

    //insert child nodes of A
    Tree<char>* B = tree.addChild('B');
    Tree<char>* C = tree.addChild('C');
    Tree<char>* D = tree.addChild('D');
    Tree<char>* E = tree.addChild('E');
    Tree<char>* F = tree.addChild('F');
    Tree<char>* G = tree.addChild('G');
```

```cpp

    //child of D
    Tree<char>* H = D->addChild('H');

    //children of E
    Tree<char>* I = E->addChild('I');
    Tree<char>* J = E->addChild('J');

    //children of F
    Tree<char>* K = F->addChild('K');
    Tree<char>* L = F->addChild('L');
    Tree<char>* M = F->addChild('M');

    //child of G
    Tree<char>* N = G->addChild('N');

    //children of J
    Tree<char>* P = J->addChild('P');
    Tree<char>* Q = J->addChild('Q');

    //traversing
    std::cout<< "preorder: ";
    tree.preOrder();
    std::cout<<std::endl;

    std::cout<< "postorder: ";
    tree.postOrder();
    std::cout<<std::endl;

    return 0;
}
```

HEADER FILE

```cpp
#ifndef TREE_H
#define TREE_H

template <typename T>
class Tree
{
private:
    // create a key
    T key;
    Tree<T>* firstChild;
    Tree<T>* nextChild;

public:
    // constructor
    Tree(T value);

    // add child
    Tree<T>* addChild(T value);

    // traversing (TRAVERSING TECHNIQUES)
    void preOrder();
    void postOrder();
```

CMakeLists.txt | main.cpp | BST.h

```cpp
// Constructor
template <typename T>
Tree<T>::Tree(T value)
{
    key = value;
    firstChild = nullptr;
    nextChild = nullptr;
}

// Add child
template <typename T>
Tree<T>* Tree<T>::addChild(T value)
{
    Tree<T>* newNode = new Tree<T>(value);

    // if this is the first child, attach it directly
    if (firstChild == nullptr){
        firstChild = newNode;
    } else {
        // otherwise walk the sibling list to the end
        Tree<T>* temp = firstChild;
        while (temp->nextChild != nullptr){
            temp = temp->nextChild;
        }
        temp->nextChild = newNode;
    }
    return newNode;
}

template <typename T>
void Tree<T>::preOrder(){
    std::cout << key << " ";
    // whole subtree of the first child (includes its siblings, i.e. my other children)
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    // move on to my own next sibling
    if(nextChild != nullptr){
        nextChild->preOrder();
    }
}
template <typename T>
void Tree<T>::postOrder(){
    // finish every child's whole subtree first, in order
    Tree<T>* child = firstChild;
    while(child != nullptr){
        child->postOrder();
        child = child->nextChild;
    }
    // then visit this node
    std::cout << key << " ";
}
```

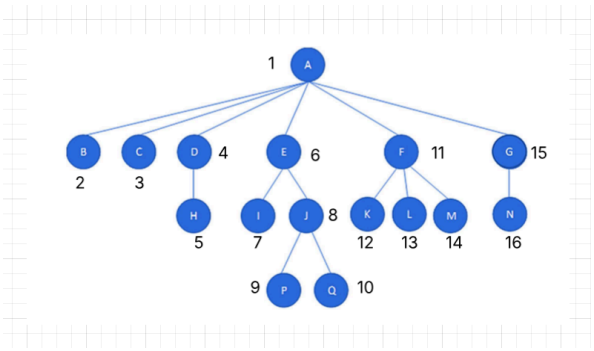
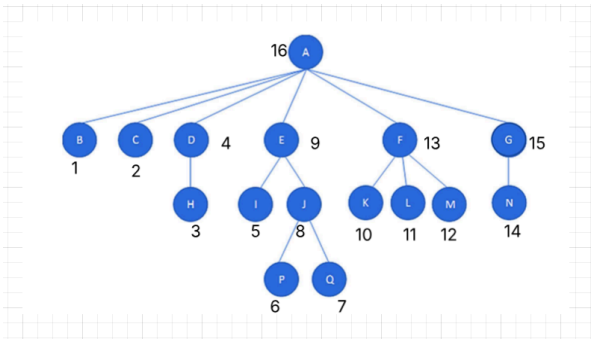
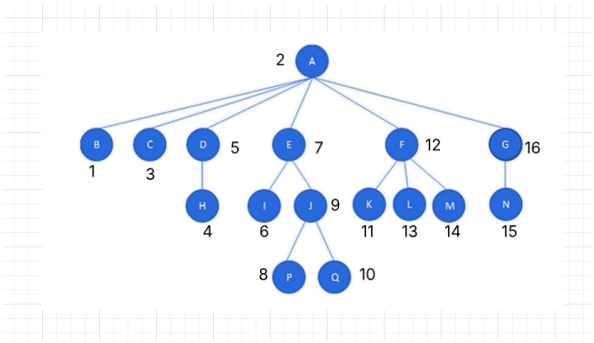
Tree<T>::postOrder

Contains the code for the main and header files for the binary tree program containing given letters A to Q. Code has the same structure compared to that of the code given during lecture, however the difference lies in the two fixed pointers that now allows us to have any number of children in order to allow us to have any number or

nodes of children to allow for the number of characters which are the letters A to Q.

TASK 2 (TABLE 9.2)

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post- order traversal	
TASK 3 (TABLE 9.3)	
Pre-order	
Post-order	
In-order	

TASK 3.2 (TABLE 9.4)

Pre-order

```
67
68  // Preorder
69  template <typename T>
70  void BinarySearchTree<T>::preOrder(){
71      std::cout << key << " ";
72
73      // left
74      if(left != nullptr){
75          left->preOrder();
76      }
77
78      // right
79      if(right != nullptr){
80          right->preOrder();
81      }
82  }
83
```

```
preorder: 20 10 5 70 60 50 80
```

Output of the function for the function of pre-order traversing in the binary tree with the given characters of letters A to Q. As observed, the print function is implemented BEFORE the if condition and the pointers.

Post-order

```
1   // Postorder
2   template <typename T>
3   void BinarySearchTree<T>::postOrder(){
4
5       // go to left
6       if(left != nullptr){
7           left->postOrder();
8       }
9
10      // go to right
11      if(right != nullptr){
12          right->postOrder();
13      }
14
15      std::cout << key << " ";
16  }
17
```

```
postorder: 5 10 50 60 80 70 20
```

Output of the function for the function of post-order traversing in the binary tree with the given characters of letters A to Q. As observed, the print function is implemented AFTER the if condition and the pointers.

In-order

```
// Inorder
template <typename T>
void BinarySearchTree<T>::inOrder(){

    // go to left
    if(left != nullptr){
        left->inOrder();
    }

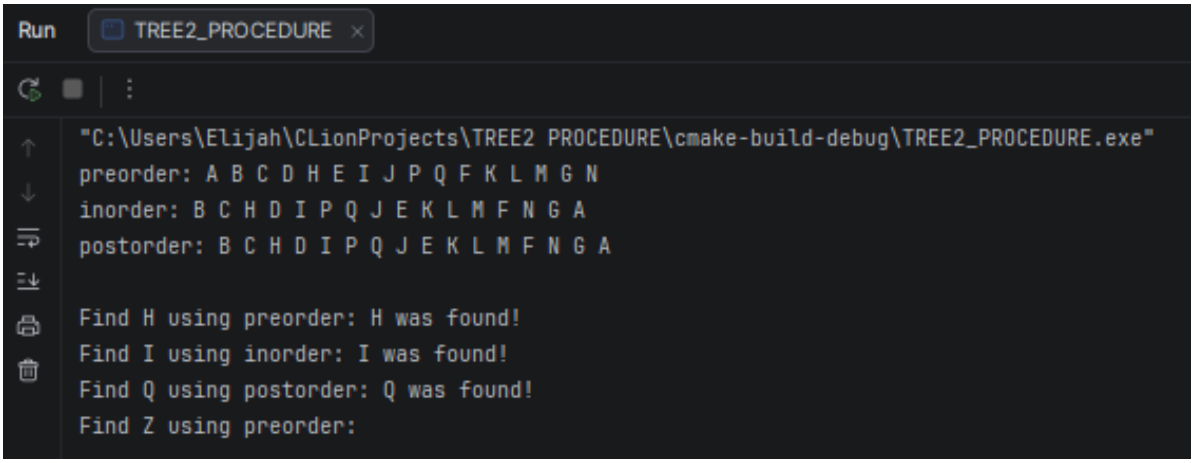
    std::cout << key << " ";

    // go to right
    if(right != nullptr){
        right->inOrder();
    }
}
```

```
inorder: 5 10 20 50 60 70 80
```

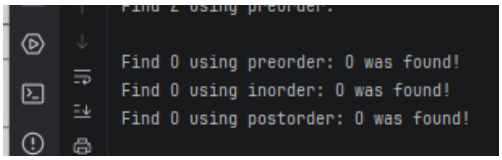
Output of the function for the function of pre-order traversing in the binary tree with the given characters of letters A to Q. As observed, the print function is implemented at the middle of the two if conditions and the pointers during trraversing.

TASK 3.3 (TABLE 9.5)



Output of the newly added function called findData with two parameters of Choice and key that tells the program which traversal method for any letter we want to search for, with the key being the value of what we actually want to find using any traversal method.

TASK 3.4 (TABLE 9.6)



Contains the output of the task of adding a new leaf node containing the data value of "O" and making the node a child of node G. Wherein we will find the said "O" using the findData function created in #3. As we can observe, with the use of different traversal techniques we were able to find the value of "O" as using any traversal method does not necessarily change its value and eventually any traversal method will be able to reach the node "O".

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

CMakeLists.txt | main.cpp | BST.h

```cpp
#include <iostream>
#include "BST.h"

int main() {
    // Create the root node with the first value in the list
    BinarySearchTree<int> tree( value: 2);

    // Insert remaining values: 3, 9, 18, 0, 1, 4, 5
    tree.insert( value: 3);
    tree.insert( value: 9);
    tree.insert( value: 18);
    tree.insert( value: 0);
    tree.insert( value: 1);
    tree.insert( value: 4);
    tree.insert( value: 5);

    // Traversing
    std::cout << "preorder: ";
    tree.preOrder();
    std::cout << std::endl;

    std::cout << "inorder: ";
    tree.inOrder();
    std::cout << std::endl;

    std::cout << "postorder: ";
    tree.postOrder();
    std::cout << std::endl;

    // Searching
    std::cout << "Find 4 using preorder: ";
    tree.findData( CHOICE: 1, KEY: 4);
    std::cout << std::endl;

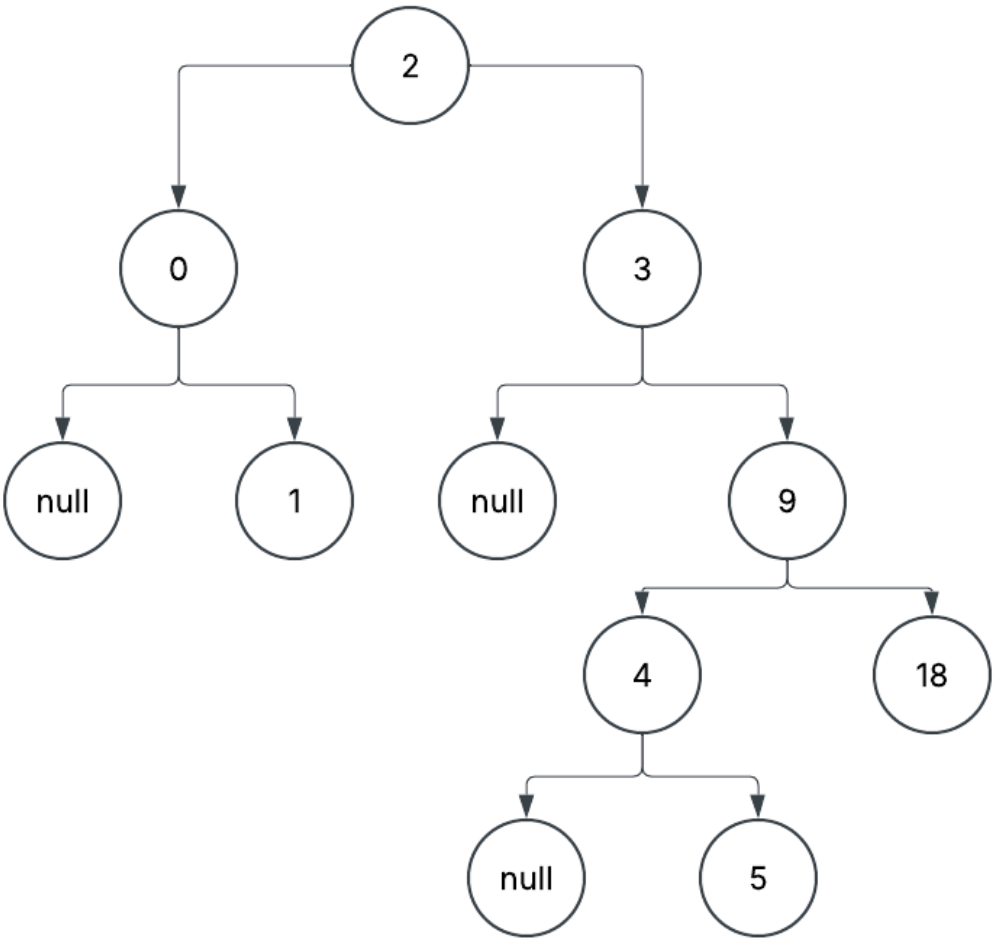
    std::cout << "Find 18 using inorder: ";
    tree.findData( CHOICE: 2, KEY: 18);
    std::cout << std::endl;

    std::cout << "Find 10 using postorder: ";
    tree.findData( CHOICE: 3, KEY: 10);
    std::cout << std::endl;

    return 0;
}
```

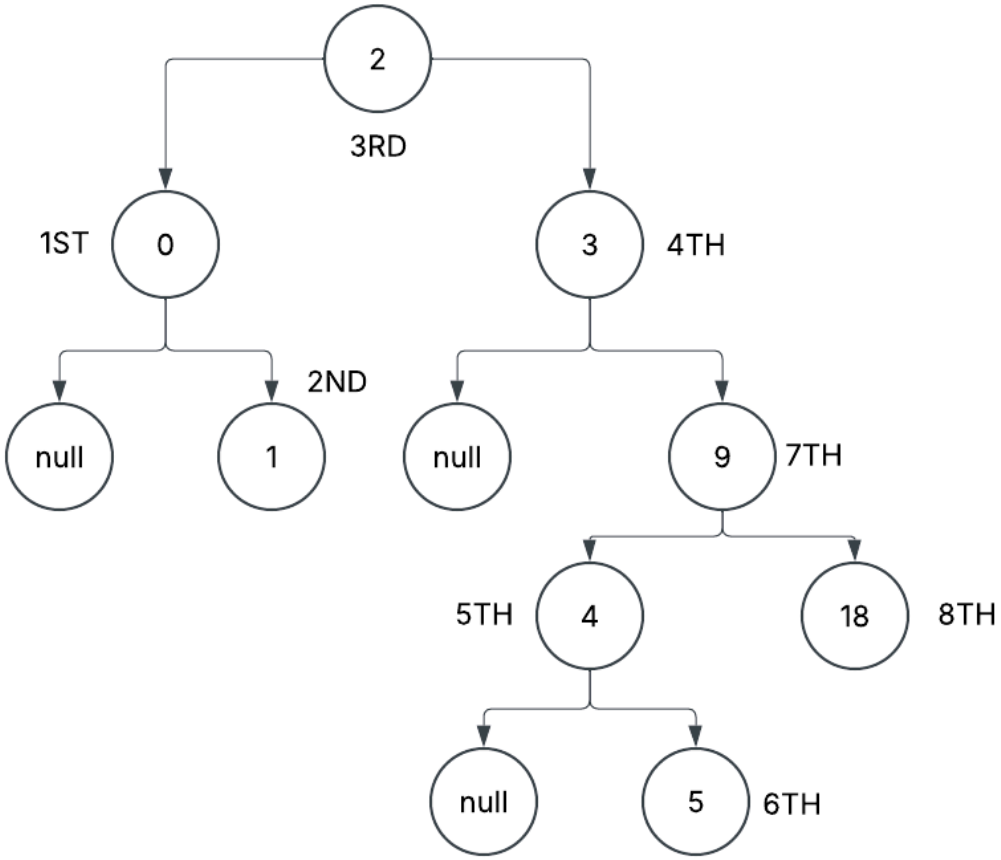
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

BINARY SEARCH TREE
2, 3, 9, 18, 0, 1, 4, 5.



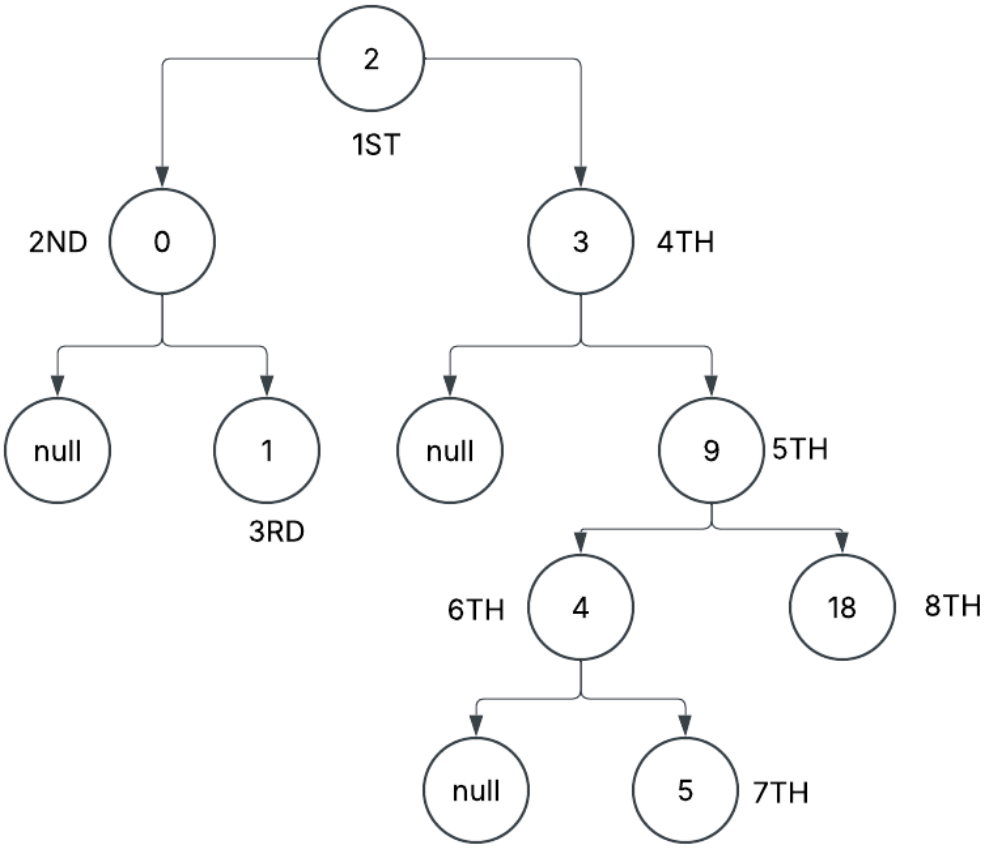
IN ORDER TRAVERSAL

BINARY SEARCH TREE
2, 3, 9, 18, 0, 1, 4, 5.
IN ORDER

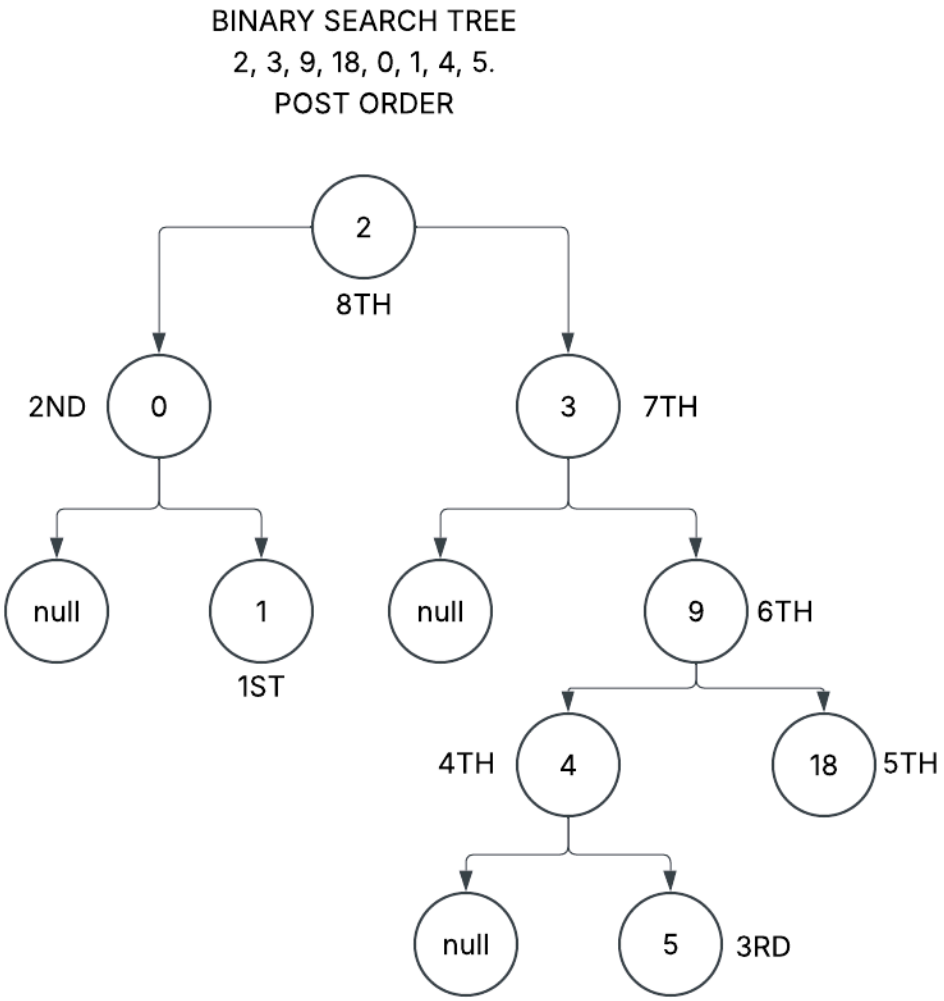


PRE ORDER TRAVERSAL

BINARY SEARCH TREE
2, 3, 9, 18, 0, 1, 4, 5.
PRE ORDER



POST ORDER TRAVERSAL



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

OUTPUT

```
C:\Users\Elijah\CLionProjects\TREE2\cmake-build-debug\TREE2.exe
preorder: 2 0 1 3 9 4 5 18
inorder: 0 1 2 3 4 5 9 18
postorder: 1 0 5 4 18 9 3 2
Find 4 using preorder: 4 was found!
Find 18 using inorder: 18 was found!
Find 10 using postorder:

Process finished with exit code 0
```

In-order traversal was performed with the In-order function present within the header file, BST.h.

Pre-order Traversal

```
68  // Preorder
69  template <typename T>
70  void BinarySearchTree<T>::preOrder(){
71      std::cout << key << " ";
72
73      // left
74      if(left != nullptr){
75          left->preOrder();
76      }
77
78      // right
79      if(right != nullptr){
80          right->preOrder();
81      }
82  }
```

```
C:\Users\Elijah\CLionProjects\TREE2\cmake-build-debug\TREE2.exe
preorder: 2 0 1 3 9 4 5 18
```

Post-order Traversal

```
// Postorder
template <typename T>
void BinarySearchTree<T>::postOrder(){

    // go to left
    if(left != nullptr){
        left->postOrder();
    }

    // go to right
    if(right != nullptr){
        right->postOrder();
    }

    std::cout << key << " ";
}
```



OBSERVATIONS

As observed, the output of the post and pre order traversal are different. With the post order starting at the value of 1 and pre order starting at the value of 2. This is because of how each method traverses within the given binary tree.

CONCLUSION/LEARNINGS/ANALYSIS

The activity focuses on learning the fundamentals and implementation of a binary tree with different given scenarios and values. By learning how a binary tree functions, it allows us to create and implement a binary tree with different root values while also learning how to use the different traversal methods that allows us to access child nodes in different order. In addition, learning the fundamentals of a binary tree allows us to learn how to also implement searching for different values in a binary tree.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code correction, explanation of concepts, grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Code correction, explanation of concepts, and grammar correction to complete the work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.